

EduStack Enterprise

Complete Technical Project Documentation & Architecture Blueprint

Your Ultimate Computer Science & Engineering Hub

Full Stack Software Architecture | Node.js Express | Python FastAPI | MongoDB | Security | AI RAG Engine

Project Name:	EduStack - Your Ultimate Computer Science Hub
Author & Role:	Shubham Kumar Full-Stack Developer & Architect NIT Patna
Repository:	github.com/ShubhamKumar968/EduStack--Your-Ultimate-Computer-Science-Hub
Backend Core:	Node.js 18 LTS + Express.js 4 (REST API Monolith + Static Server)
ML Microservice:	Python 3.11 + FastAPI + Uvicorn (Google Gemini + LightRAG)
Database Stack:	MongoDB Atlas Managed Cluster + Mongoose 8 ODM + connect-mongodb-session
Security Engine:	Stateless JWT httpOnly Cookies, bcrypt-12, Helmet, NoSQL Sanitize, Rate Limiter
Cloud & Media:	Render.com Web Services + Cloudinary CDN Media Storage + Nodemailer SMTP
Version & Date:	v1.0.0 Enterprise Production Specification - August 2026

Official Comprehensive Engineering Documentation & Technical Architecture Reference

Table of Contents — Complete System Specification

1.	Project Overview & Problem Statement	Centralized academic repository, AI tutor, live DSA tracker, role access
2.	System Architecture & Hybrid Infrastructure	Node.js monolith + Python FastAPI microservice proxy architecture
3.	Tech Stack Deep Dive	Frontend, Backend, ML/AI, Databases, and Security dependencies
4.	Backend - Node.js + Express	10-middleware pipeline order, asyncHandler HOF, JSON response envelope
5.	Database Design - MongoDB + Mongoose	User, OTP TTL index, Subject, Resource, Payment, and Notification schemas
6.	Authentication System	Local OTP verification, JWT httpOnly cookies, Google OAuth 2.0, isAuth
7.	Security Implementation	17-layer defense in depth, HMAC timingSafeEqual, NoSQL sanitization
8.	Payment Gateway - Razorpay	3-step lifecycle, server-side HMAC-SHA256 verification, fraud protection
9.	ML / AI Microservice - Python + FastAPI	Google Gemini 1.5/2.0 Flash, LightRAG retrieval, fallback models, PDF OCR
10.	Frontend Architecture	Multi-page application, partials.js engine, auth guard, CSS variables theme
11.	Responsiveness & UI Design	Breakpoints, circular avatar across all devices, dark mode, glassmorphism
12.	API Design Patterns	REST conventions, HTTP status codes, requireRole RBAC, global error handler
13.	Cloud Services & File Uploads	Multer memoryStorage + Base64 URI + Cloudinary CDN, Nodemailer SMTP
14.	DSA Sheet - Live Google Sheet Sync	3-layer data strategy: memory cache, live fetch, disk JSON fallback, parser
15.	Notification System	Broadcast model with per-user readBy array & \$addToSet deduplication
16.	Deployment Architecture	Render.com 2 services, trust proxy, Graceful Shutdown handlers
17.	Security Analysis	13 implemented security controls and future enhancement roadmap
18.	Interview Q&A - Full Coverage	Auth, Database, ML/AI, Frontend, and System Architecture interview questions
19.	Summary Cheat Sheet	Complete multi-category reference table covering the entire EduStack stack

Project Overview & Problem Statement

Centralized academic repository, AI tutor, live DSA tracker, role access, and student pain points

1.1 What is EduStack?

EduStack is a production-grade full-stack web application engineered for engineering and computer science students (primarily at NIT Patna). It resolves content fragmentation across telegram channels, unindexed Google Drive folders, Reddit threads, and scattered websites by consolidating all study resources into a secure, unified platform.

Core Platform Capabilities

- **Subject Repository:** Curated lecture notes, previous year question papers (PYQs), and YouTube playlists organized by semester.
- **AI Hub:** Google Gemini-powered AI Exam Tutor, automated PYQ solution generator, PDF summarizer, and automated quiz generator.
- **Premium DSA Sheet:** 450+ hand-curated competitive programming problems synced live from Google Sheets with in-memory caching.
- **Role-Based Access Control:** Granular permissions for Students, Contributors, and Administrators.
- **Razorpay Payment Gateway:** Secure transactional premium subscription with HMAC-SHA256 verification (Rs.5 demo simulation).
- **Notification Engine:** System-wide admin broadcasts with per-user read state tracking using MongoDB arrays.
- **Personalized Learning:** Favourites, subject enrollments, and customized multi-accent themes (36 combinations).

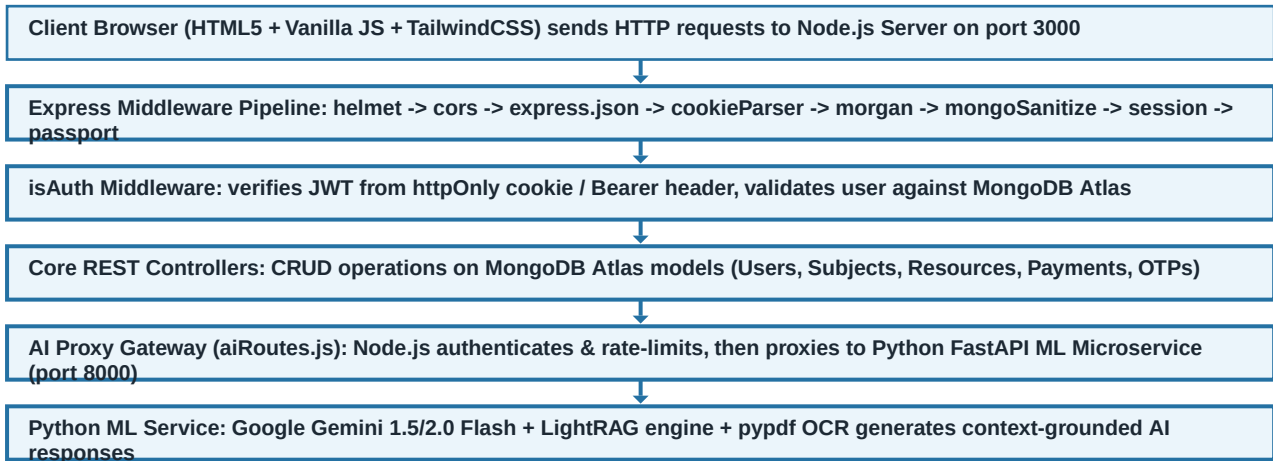
System Architecture & Hybrid Infrastructure

Node.js Express monolith + Python FastAPI microservice proxy architecture

2.1 Architectural Topology

EduStack uses a deliberate hybrid architecture: a Node.js Express monolith handles all core business logic (auth, subjects, resources, payments, DSA sheet), while a separate Python FastAPI microservice handles AI/ML processing. This is a common production pattern at companies like Stripe, Shopify, and Flipkart.

FLOW: EduStack Request Flow Topology



CHAPTER 3

Tech Stack Deep Dive

Frontend, Backend, ML/AI, Databases, and Security dependencies

3.1 Technology Matrix		
Layer	Technology	Role & Rationale
Frontend UI	HTML5 + Vanilla JS + TailwindCSS	Zero framework overhead, instant initial loads, full DOM control without complex build chains
Icons & Fonts	Font Awesome 6 + Google Fonts Outfit/ Inter	Clean typography, responsive icons via CDN
Backend Runtime	Node.js 18 LTS + Express.js 4	Single-threaded async I/O capable of handling 10,000+ concurrent connections efficiently
Database Layer	MongoDB Atlas + Mongoose 8 ODM	Flexible BSON document schemas, auto-timestamps, TTL indexes, and virtual fields
Auth & Security	JWT + bcryptjs (12 rounds) + Passport.js	Stateless tokens in httpOnly cookies + Google OAuth 2.0 Authorization Code Grant
Payment System	Razorpay SDK + Node Crypto	HMAC-SHA256 signature verification with timingSafeEqual fraud prevention
Media & CDN	Multer + Cloudinary CDN	Memory storage buffer to base64 URI direct CDN streaming without server disk consumption
AI / ML Engine	Python 3.11 + FastAPI + Uvicorn + Gemini	Google Gemini 1.5/2.0 Flash + LightRAG store + pypdf dual-strategy PDF OCR
Deployment	Render.com Web Services (2 services)	Independent scaling of Node.js API and Python ML worker with auto-HTTPS and trust proxy

Backend - Node.js + Express Architecture

10-middleware pipeline order, asyncHandler HOF, JSON response envelope

4.1 Middleware Pipeline Order (Order Matters)

- **1. helmet({ contentSecurityPolicy: false })**: Security headers on ALL responses before any data is sent.
- **2. cors({ origin: fn, credentials: true })**: CORS preflight (OPTIONS) requests handled before body parsing.
- **3. app.set("trust proxy", 1)**: Enables X-Forwarded-For header trust from Render.com load balancer for accurate rate limiting and session cookies.
- **4. express.urlencoded({ extended: true })**: Parses URL-encoded form submissions.
- **5. express.json({ limit: "1mb" })**: Parses JSON body with 1MB ceiling preventing large payload DoS attacks.
- **6. cookieParser()**: Populates req.cookies before authentication checks.
- **7. morgan("combined" / "dev")**: HTTP access logger registered after body parser to log payload sizes.
- **8. mongoSanitize()**: Strips \$ and . characters from user inputs preventing NoSQL injection before reaching routes.
- **9. session() + passport.initialize() + passport.session()**: State persistence for OAuth 2.0 flow.
- **10. Routes & Error Handler**: API routers mounted; 4-parameter errorHandler registered LAST.

4.2 asyncHandler & Response Envelope Pattern

JavaScript / Node.js / Python

```
// utils/asyncHandler.js - Higher-Order Function wrapper
const asyncHandler = (fn) => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);

// utils/apiResponse.js - Standardized JSON envelope
const sendSuccess = (res, message, data = {}, statusCode = 200) =>
  res.status(statusCode).json({ success: true, message, data, timestamp: new
Date().toISOString() });
const sendError = (res, message, statusCode = 500, errors = []) =>
  res.status(statusCode).json({ success: false, message, errors, timestamp: new
Date().toISOString() });
```

Database Design - MongoDB + Mongoose

User, OTP TTL index, Subject, Resource, Payment, and Notification schemas

5.1 User Model & Schema Annotations

JavaScript / Node.js / Python

```
// models/user.js - Security-First User Schema
const userSchema = new mongoose.Schema({
  firstName: { type: String, required: true, trim: true, maxlength: 50 },
  lastName: { type: String, required: true, trim: true, maxlength: 50 },
  email: { type: String, required: true, unique: true, lowercase: true, trim: true },
  password: { type: String, select: false, minlength: 6 }, // select:false prevents password leak
  googleId: { type: String, default: null, index: true }, // O(log n) B-Tree index
  role: { type: String, enum: ["user", "student", "contributor", "admin"], default: "user" },
  isVerified: { type: Boolean, default: false },
  isPremium: { type: Boolean, default: false },
  avatar: { type: String, default: "default-avatar.png" },
  branch: { type: String, default: "CSE" },
  semester: { type: Number, min: 1, max: 8, default: 1 },
}, { timestamps: true });

userSchema.pre("save", async function(next) {
  if (!this.isModified("password") || !this.password) return next();
  if (!this.password.startsWith("$2a$") && !this.password.startsWith("$2b$")) {
    this.password = await bcrypt.hash(this.password, 12);
  }
  next();
});
```

OTP TTL Index & Payment Schemas

- **OTP Schema:** { email: String, code: String, expiresAt: Date } with TTL index { expiresAt: 1 }, { expireAfterSeconds: 0 } - MongoDB auto-deletes expired records with zero cron overhead.
- **Payment Schema:** { user: ObjectId, razorpayOrderId: String (unique), razorpayPaymentId: String, razorpaySignature: String, amount: Number (paise), status: enum["created", "paid", "failed"] }.
- **Resource Schema:** Compound index { subject: 1, type: 1 } optimizes queries filtering notes/PYQs per subject.
- **Notification Schema:** { title: String, message: String, readBy: [{ type: ObjectId, ref: "User" }] } - embedded read tracking.

Authentication System

Local OTP verification, JWT httpOnly cookies, Google OAuth 2.0, isAuth gatekeeper

6.1 Dual Authentication Flow

- **Local Registration:** Password hashed with bcrypt (12 rounds). 6-digit cryptographic OTP emailed via Nodemailer. Account remains isVerified: false until verified.
- **OTP Verification:** Checks MongoDB OTP record. On match, deletes OTP immediately (single use), sets isVerified: true, and fires welcome email.
- **Login:** User.findOne({ email }).select("+password"). Compares via bcrypt.compare(). Returns JWT in httpOnly cookie.
- **Google OAuth 2.0:** Passport.js Authorization Code Grant. Pre-verifies email, checks ADMIN_EMAILS env list, sets JWT cookie on redirect.

6.2 isAuth Middleware Implementation

JavaScript / Node.js / Python

```
// middlewares/isAuth.js - Dual Token Source Verifier
const isAuth = async (req, res, next) => {
  try {
    let token = req.headers.authorization?.startsWith("Bearer ")
      ? req.headers.authorization.split(" ")[1]
      : req.cookies?.edustack_token;

    if (!token) return sendError(res, "Access denied. Please log in.", 401);

    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(decoded.id).select("-password");
    if (!user) return sendError(res, "User not found.", 401);
    if (!user.isVerified) return sendError(res, "Account not verified.", 403);

    req.user = user; // Attached for downstream controllers
    next();
  } catch (err) {
    return sendError(res, "Invalid or expired session token.", 401);
  }
};
```


CHAPTER 7

Security Implementation

17-layer defense in depth, HMAC timingSafeEqual, NoSQL sanitization

7.1 17 Layers of Defense in Depth

Layer	Implementation	Threat Mitigated
1. Security Headers	Helmet.js (15+ headers)	Clickjacking, MIME sniffing, XSS, HSTS
2. CORS Whitelist	Dynamic origin callback	Unauthorized cross-origin API calls
3. httpOnly Cookies	JWT in httpOnly cookie	XSS JavaScript token theft
4. Password Hashing	bcryptjs with 12 salt rounds	Rainbow tables, GPU brute force
5. NoSQL Sanitization	express-mongo-sanitize	MongoDB \$gt/\$where injection bypasses
6. Rate Limiting	express-rate-limit (30/10min on AI)	DDoS, credential stuffing, scraping
7. Token Expiry	7-day expiration window	Compromised token lifetime limit
8. HMAC Verification	crypto.createHmac("sha256")	Forged payment confirmations
9. Constant-Time Compare	crypto.timingSafeEqual()	Side-channel timing attacks
10. Admin via Env	ADMIN_EMAILS in .env only	Git repo leak privilege escalation
11. select: false	Password excluded by default	Accidental credential exposure in JSON
12. OTP TTL Index	5-minute MongoDB expiration	OTP brute force attack window
13. OTP Single-Use	deleteOne() immediately on check	Replay attacks
14. Anti-Enumeration	Identical response for unknown emails	User email harvesting
15. Body Payload Limit	express.json({ limit: "1mb" })	Large payload DoS
16. Trust Proxy	app.set("trust proxy", 1)	IP spoofing behind load balancers
17. Graceful Shutdown	SIGTERM/SIGINT server.close()	In-flight request data loss

Payment Gateway - Razorpay Integration

3-step lifecycle, server-side HMAC-SHA256 verification, fraud protection

8.1 3-Step Payment Architecture

- **Step 1 (Order Creation):** Client requests POST /api/payments/create-order. Node.js backend calls Razorpay SDK with amount: 500 (paise = Rs.5). Stores Payment record with status: "created". Returns orderId and keyId.
- **Step 2 (Client Checkout):** Frontend launches Razorpay Modal. User completes UPI/Card payment. Razorpay returns { razorpayOrderId, razorpayPaymentId, razorpaySignature }.
- **Step 3 (HMAC Verification):** Client sends receipt to POST /api/payments/verify. Server recomputes HMAC-SHA256 and verifies using crypto.timingSafeEqual(). If valid, marks Payment as "paid" and sets user.isPremium = true.

JavaScript / Node.js / Python

```
// services/razorpayService.js - Fraud-Proof Payment Verification
const verifyPaymentSignature = (orderId, paymentId, signature) => {
  const message = `${orderId}|${paymentId}`;
  const expected = crypto
    .createHmac("sha256", process.env.RAZORPAY_KEY_SECRET)
    .update(message)
    .digest("hex");

  try {
    return crypto.timingSafeEqual(
      Buffer.from(expected, "hex"),
      Buffer.from(signature, "hex")
    );
  } catch { return false; }
};
```

ML / AI Microservice - Python + FastAPI

Google Gemini 1.5/2.0 Flash, LightRAG retrieval, fallback models, PDF OCR

9.1 Architecture & RAG Engine

- **Microservice Isolation:** Python FastAPI service runs independently with Google Generative AI SDK, pypdf, and Uvicorn ASGI server.
- **LightRAG Store:** Retrieves top-K course chunks using TF-IDF word frequency scoring. Augments Gemini prompt with retrieved context to prevent hallucination.
- **Self-Healing Model Fallback:** Queries `genai.list_models()` dynamically and waterfalls through Gemini 1.5 Flash -> 2.0 Flash -> Pro if quota or deprecation errors occur.
- **Dual-Strategy PDF Processing:** Uses pypdf for text-selectable documents (>50 chars). Uses Gemini Multimodal Vision OCR for scanned image PDFs.

JavaScript / Node.js / Python

```
# ml_services/main.py - PDF Dual Processing Strategy
@app.post("/api/pdf/summarize")
async def summarize_pdf(file: UploadFile = File(...)):
    pdf_bytes = await file.read()
    text = extract_text_from_pdf_bytes(pdf_bytes)
    if len(text) > 50:
        contents = [f"{prompt}\n\nDocument Text:\n{text[:10000]}"]
    else:
        contents = [{"mime_type": "application/pdf", "data": pdf_bytes}, prompt]
    response = model.generate_content(contents)
    return {"success": True, "summary": response.text}
```

Frontend Architecture & partials.js Engine

Multi-page application, partials.js engine, auth guard, CSS variables theme

10.1 partials.js - The Global Frontend Engine

EduStack is an MPA (Multi-Page Application) where every HTML file includes partials.js. This 91KB client script handles dynamic navbar injection, auth state verification (/api/auth/me caching in window.currentUser), theme switching, notifications, slide-in mobile drawers, and auth guard modals.

JavaScript / Node.js / Python

```
// client/assets/js/partial.js - Auth Guard Pattern
window.requireAuth = function(action, label) {
  if (window.currentUser) {
    action();
  } else {
    fetch("/api/auth/me", { credentials: "include" })
      .then(r => r.json())
      .then(d => {
        if (d?.success && d?.data?.user) {
          window.currentUser = d.data.user;
          action();
        } else {
          window.showAuthModal(label || "access this feature");
        }
      });
  }
};
```

Responsiveness & UI Design

Breakpoints, circular avatar across all devices, dark mode, glassmorphism

11.1 Adaptive Breakpoints & Glassmorphism

Breakpoint	Width	Layout Behavior
xs (Mobile)	< 480px	Circular 36px profile pill avatar only, hamburger drawer menu
sm (Small Mobile)	480 - 639px	AI Tutor quick button and Sign Up CTA appear
md (Tablet)	640 - 1023px	Profile pill expands to show user name + role badge
lg (Desktop)	1024 - 1279px	Full desktop horizontal navigation bar with all category links
xl (Large Desktop)	>= 1280px	Complete navbar + dedicated secondary action buttons

- **Glassmorphism:** Sticky navbar uses backdrop-blur-xl bg-white/90 dark:bg-[#181818]/90 with subtle borders.
- **Theme Engine:** Sets CSS custom properties (--brand, --brand-rgb) dynamically on :root supporting 36 theme combinations.
- **Dark Mode:** Persistent class "dark" on <html> activates Tailwind dark: utility classes with localStorage sync.

API Design Patterns

REST conventions, HTTP status codes, requireRole RBAC, global error handler

12.1 RESTful Endpoint Reference

Method	Endpoint	Access Level	Purpose
POST	/api/auth/register	Public	User registration with email normalization
POST	/api/auth/verify-otp	Public	OTP verification and account activation
POST	/api/auth/login	Public	Credentials login and httpOnly cookie issuance
GET	/api/auth/me	isAuth	Get current authenticated user profile
GET	/api/subjects	Public	List CS subjects by semester with resource counts
POST	/api/subjects	isAuth + Admin	Create new subject entry
GET	/api/dsa-sheet/sync	isAuth + Premium	Get live-synced 450+ DSA problems
POST	/api/payments/create-order	isAuth	Create Razorpay order for Premium access
POST	/api/payments/verify	isAuth	Verify HMAC signature and grant Premium
POST	/api/ai/ask	isAuth + RateLimit	Proxy query to Python FastAPI ML RAG engine

Cloud Services & File Uploads

Multer memoryStorage + Base64 URI + Cloudinary CDN, Nodemailer SMTP

13.1 Zero-Disk Memory Upload Pipeline

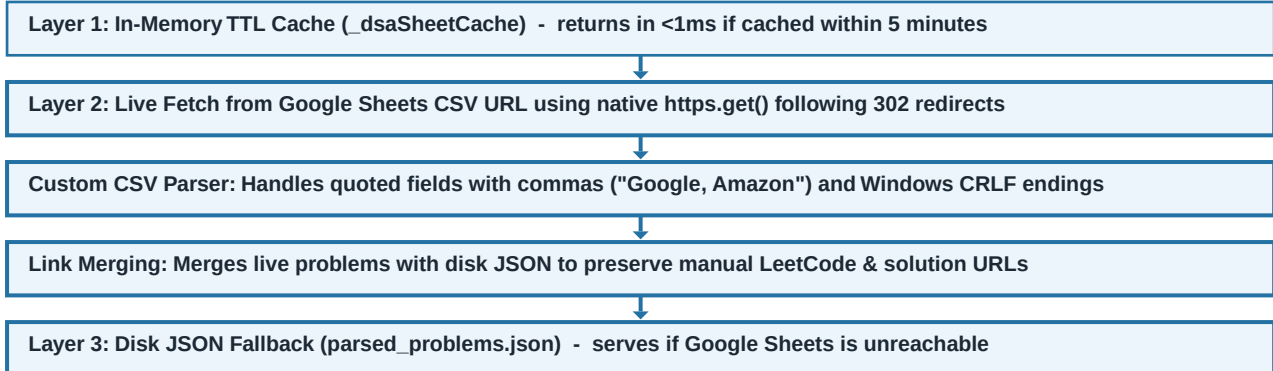
- **Multer Memory Storage:** `multer({ storage: multer.memoryStorage() })` buffers image uploads in RAM - zero server disk consumption.
- **Base64 URI Conversion:** Buffer is formatted as `data:image/jpeg;base64,...` and streamed to Cloudinary.
- **Cloudinary CDN:** Uploaded to "edustack_profiles" / "edustack_subjects" folders returning HTTPS CDN URLs.
- **Graceful Degradation:** If Cloudinary times out or fails, falls back to saving the base64 URI directly in MongoDB.
- **Nodemailer SMTP:** Sends OTP verification and welcome emails using Gmail App Passwords with fire-and-forget background delivery.

DSA Sheet - Live Google Sheet Sync

3-layer data strategy: memory cache, live fetch, disk JSON fallback, custom parser

14.1 Three-Tier Synchronization Architecture

FLOW: DSA Sheet Sync Data Strategy



Notification System

Broadcast model with per-user readBy array & \$addToSet deduplication

15.1 Broadcast Model Architecture

Instead of generating N separate notification records for N users (fan-out antipattern), EduStack creates ONE notification document. An embedded readBy array tracks user IDs who have viewed it.

JavaScript / Node.js / Python

```
// controllers/notificationController.js - Efficient Read Tracking
// Fetch with computed isRead boolean:
const notifications = await Notification.find().sort({ createdAt: -1 }).limit(30);
const data = notifications.map(n => ({
  ...n.toObject(),
  isRead: n.readBy.some(id => id.toString() === req.user._id.toString())
}));

// Mark all as read (idempotent atomic bulk update):
await Notification.updateMany(
  { readBy: { $ne: req.user._id } },
  { $addToSet: { readBy: req.user._id } } // Prevents duplicates
);
```

Deployment Architecture

Render.com 2 services, trust proxy, Graceful Shutdown handlers

16.1 Production Deployment Blueprint

- **Service 1 (Node.js API):** Render Web Service running Node 18, serving API routes and static HTML. Configured with trust proxy: 1 for correct client IP detection.
- **Service 2 (FastAPI ML):** Render Python Web Service running Uvicorn ASGI server with Gemini API keys.
- **MongoDB Atlas:** Managed 3-node replica set with automated backups and encrypted connections.
- **Graceful Shutdown:** Catches SIGTERM / SIGINT, stops accepting new requests with `server.close()`, allows active requests to finish, and closes MongoDB connections cleanly.

Security Analysis & Future Roadmap

13 implemented security controls and future enhancement roadmap

17.1 Implemented Controls vs Roadmap

Implemented Security Controls	Future Enhancement Roadmap
httpOnly + Secure + SameSite JWT cookies	CSRF double-submit tokens on state-changing forms
bcrypt with 12 salt rounds (GPU resistant)	Account lockout / exponential delay after N failed logins
HMAC-SHA256 payment verification with timingSafeEqual	Mutual TLS / API key auth between Node and Python ML
express-mongo-sanitize NoSQL injection defense	Refresh token rotation with short 15-min access tokens
Rate limiting on AI proxy (30 req / 10 min)	Admin audit logging collection for tracking mutations
MongoDB TTL auto-expiring OTP documents (5 min)	Redis distributed caching layer for multi-instance scaling

Interview Q&A - Comprehensive Technical Coverage

Auth, Database, ML/AI, Frontend, and System Architecture interview questions

Q1: How does EduStack authentication work end-to-end?

Answer:

Dual auth: Local (Email/Password + OTP + JWT) and Google OAuth 2.0. Passwords are hashed with bcrypt (12 rounds). On registration, a 6-digit OTP is saved with MongoDB TTL and emailed. Once verified, a JWT with minimal payload ({ id: userId }) is set in an httpOnly cookie. isAuth verifies JWT and re-fetches user from DB per request for instant role revocation.

- > httpOnly cookie prevents XSS token theft.
- > Re-fetching user ensures admin privilege changes take effect immediately.
- > Google OAuth links existing accounts with matching email.

Q2: Why store JWT in an httpOnly cookie instead of localStorage?

Answer:

localStorage is accessible by any JavaScript running in the browser. A single XSS vulnerability allows attackers to extract all tokens. httpOnly cookies cannot be read by JavaScript (document.cookie returns empty). The browser attaches the cookie automatically with requests.

- > Paired with Secure: true (HTTPS only) and SameSite protection.
- > Mitigates token theft risks in enterprise applications.

Q3: What is a timing attack and how do you prevent it in payment verification?

Answer:

String comparison (===) short-circuits on the first mismatched character, leaking timing information. Attackers measure response times to guess secrets char-by-char. crypto.timingSafeEqual() compares entire Buffers in constant time regardless of where mismatches occur.

- > Used in verifyPaymentSignature for Razorpay HMAC verification.
- > Essential for cryptographic signature and token comparisons.

Q4: How does the RAG engine in the AI Hub function?

Answer:

LightRAG uses TF-IDF keyword scoring to retrieve relevant subject syllabus and notes chunks from local knowledge store. It injects retrieved context into the Gemini prompt. This grounds Gemini's response in actual course materials and eliminates hallucinations.

- > Dual strategy handles text-based PDFs via pypdf and scanned PDFs via Gemini Vision OCR.
- > Dynamic model fallback catches deprecations and quota limits.

Q5: How would you scale EduStack to 1,000,000 users?

Answer:

Replace in-memory DSA cache with Redis cluster, move static assets to Cloudflare CDN, run multiple Node.js instances behind an Nginx/ALB load balancer, offload emails and PDF tasks to BullMQ queues, use MongoDB read replicas, and use Pinecone vector database for AI embeddings.

- > Stateless JWT auth scales across server instances without shared session state.
- > Cloudinary handles media offload without server load.

Summary Cheat Sheet

Complete multi-category reference table covering the entire EduStack stack

Category	Technology / Pattern	Key Implementation Detail
Authentication	JWT + bcrypt + OTP	httpOnly cookie, 12 salt rounds, 7-day expiry, select:false
OAuth SSO	Passport.js GoogleStrategy	ADMIN_EMAILS env list, links existing email accounts
Payment	Razorpay + HMAC-SHA256	crypto.timingSafeEqual, server-side verification, paise units
Database	MongoDB Atlas + Mongoose 8	TTL index for OTP (5 min), compound indexes, virtual fields
Security	Helmet + mongoSanitize + CORS	17 defense layers, trust proxy:1, 1MB body limit
AI Engine	FastAPI + Gemini + LightRAG	TF-IDF retrieval, model fallback waterfall, PDF Vision OCR
DSA Tracker	Google Sheets CSV live sync	5-min memory cache, disk JSON fallback, custom parser
File Storage	Multer + Cloudinary CDN	Memory buffer to base64 data URI, zero server disk usage
Frontend	Vanilla JS + TailwindCSS	partials.js engine, window.currentUser, 36 theme combos
Notifications	MongoDB readBy array	\$addToSet atomic deduplication, single broadcast record
Deployment	Render.com (2 services)	Independent Node.js + Python ML scaling, graceful shutdown